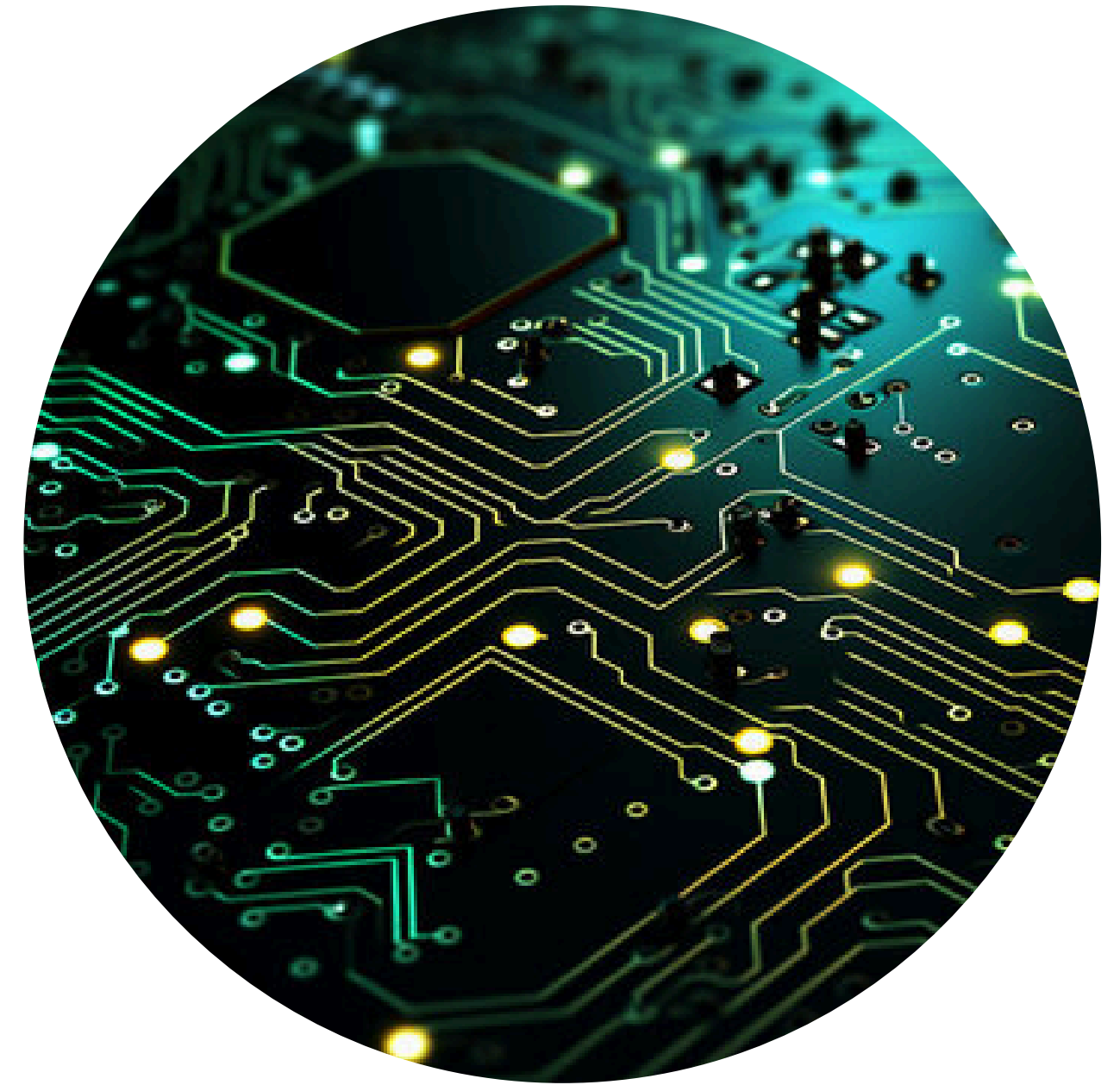


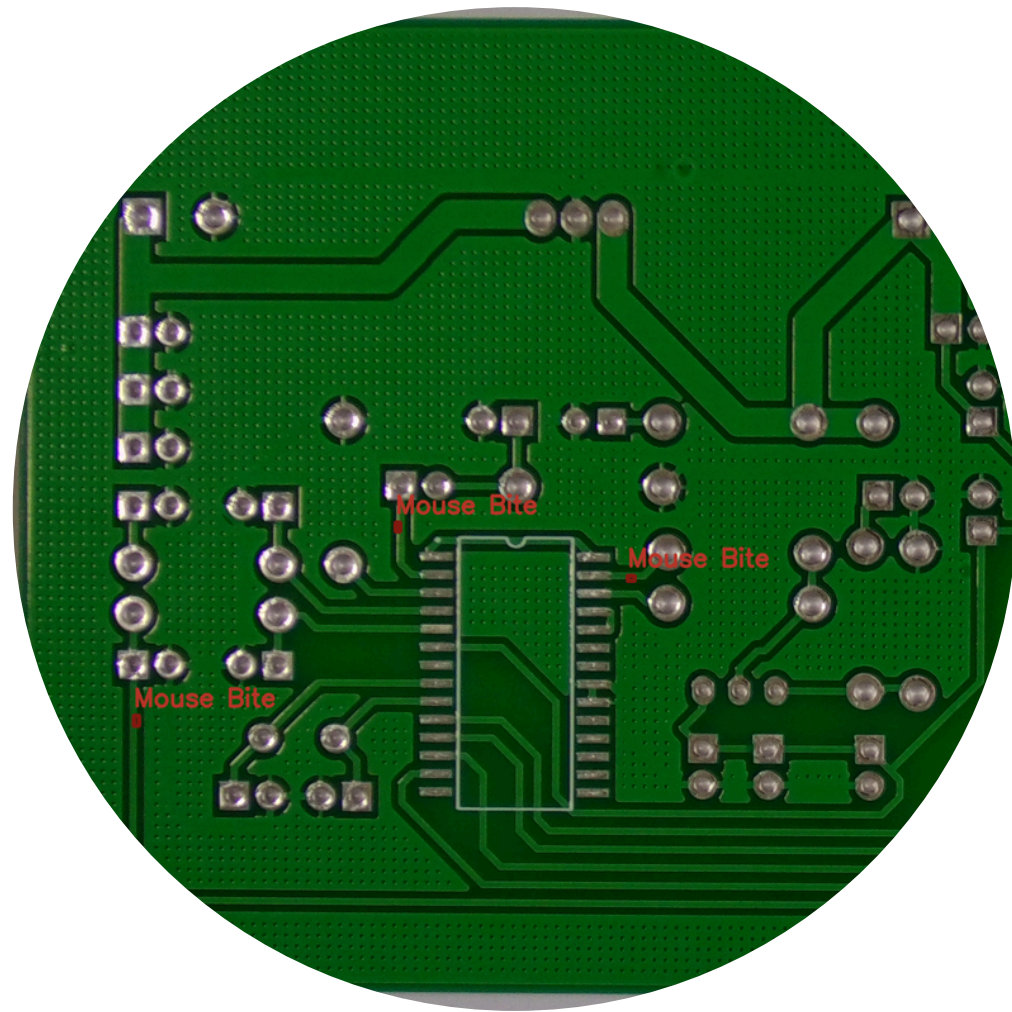
CircuitGuard - PCB Defect Detection and Classification Application



- KAVYAS



Introduction



Overview:

Printed Circuit Boards (PCBs) are prone to defects that affect the reliability and performance of electronic devices.

Manual inspection is slow, error-prone, and not scalable for mass production.

This project develops an automated defect detection system combining image processing and deep learning, specifically using EfficientNet-B4. The workflow includes image subtraction, ROI extraction, model training, evaluation, and frontend integration to deliver accurate results in real-time.

Key Features:

High accuracy (~97%) in classifying multiple defect types.

Real-time predictions via a Streamlit UI, making it user-friendly for non-technical users.

Generates annotated outputs with bounding boxes and defect labels.

Image Subtraction

Objective: Identify defect regions by comparing reference and test images.

Image Subtraction:

- Loaded reference (template) and defected PCB images.
- Aligned images to ensure pixel-level correspondence.
- Subtracted the reference from the defected image to highlight areas of difference, representing potential defects.

Thresholding and Enhancement:

- Applied Gaussian blur to reduce noise in the difference map.
- Used Otsu's thresholding to convert the difference map into a clear binary mask.
- The processed image shows highlighted defect regions, ready for ROI extraction.

ROI Extraction

Objective: Extract defect regions to create a dataset for model training.

Contour Detection:

- Applied OpenCV contour detection to find connected regions in the thresholded difference map.
- Filtered out small and noisy contours to ensure only relevant defects were captured.
- Excluded contours touching the image borders to avoid background or alignment noise (not part of the PCB).

Bounding Boxes and Cropping:

- Generated bounding boxes around valid contours.
- Cropped ROIs from the defected image corresponding to each bounding box.

Dataset Preparation:

- Labeled ROIs according to defect types (missing hole, mouse bite, short, etc.).
- Stored ROI information in a CSV file, mapping each ROI image to its label.

Model Training

Objective: Train EfficientNet-B4 model to classify defect types.

Data Preparation:

- Resized ROIs to 128×128 pixels for consistency.
- Applied normalization to standardize pixel values and improve model stability during training.

Model Architecture:

- Used EfficientNet-B4, known for high accuracy and efficient computation.
- Model configured for 6-class classification corresponding to defect types.

Training Process:

- Split dataset into training and test sets (70% training, 30% testing)
- Optimized using Adam optimizer and Cross-Entropy Loss.
- Monitored training loss and validation accuracy over epochs.
- Early stopping applied to avoid overfitting.

Model Testing

Objective: Evaluate the trained model on unseen test ROIs.

Testing Process:

- Used the reserved test dataset (30%) to evaluate how well the trained EfficientNet model generalizes to unseen defects.
- Ran predictions on the test ROIs to compute overall accuracy.
- Calculated class-wise precision, recall, and F1-score.

Performance Analysis:

- Observed class-wise metrics to identify strengths and areas for improvement.
- Checked predictions against true labels to ensure reliable classification across all defect categories.

Outcome:

- Model achieved ~96.9% overall accuracy.
- Predictions matched well with true labels, with minimal false positives or negatives.

Frontend Development

Objective: Create an interactive and user-friendly interface for PCB defect detection.

Streamlit UI:

- Users can upload PCB images and view results in real-time, with outputs annotated with bounding boxes and defect labels.
- Provides an option to download processed images for documentation and future reference.
- Designed for ease of use, allowing even non-technical users to operate the system effortlessly.

Interactive and Responsive Interface:

- Uploaded images are processed and results are displayed promptly, allowing users to verify defects efficiently.
- The interface is intuitive and user-friendly, ensuring smooth navigation

Backend Inference Pipeline

Objective: Automate preprocessing, ROI extraction, and defect classification.

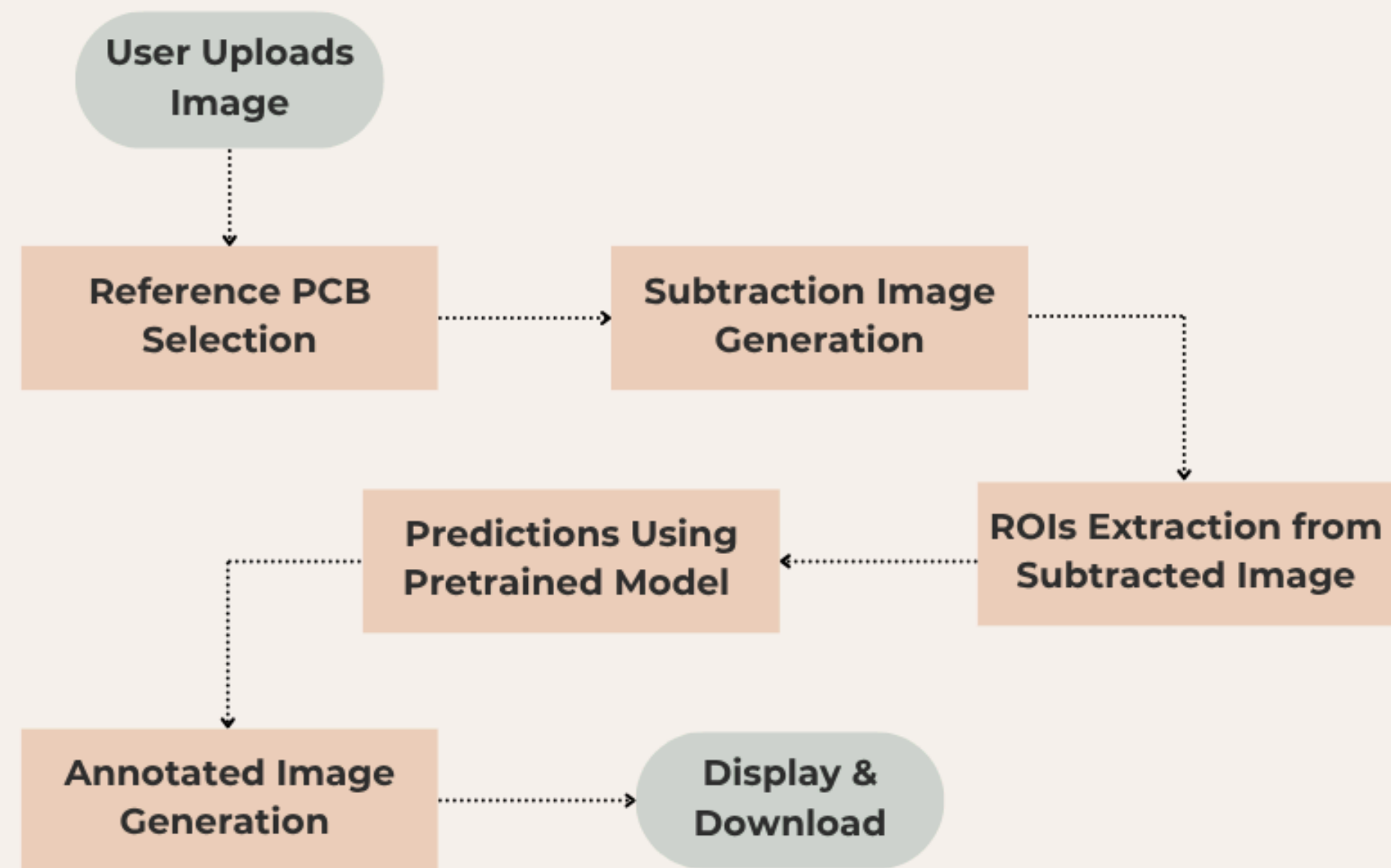
Automatic Processing:

- Uploaded PCB images are automatically compared against preloaded reference images in the backend.
- Uploaded images are preprocessed to enhance features and reduce noise.
- Contours are detected and ROIs are extracted automatically.
- Each ROI is classified using the trained EfficientNet-B4 model.

Annotated Output:

- Results display bounding boxes and defect labels clearly.
- The system generates annotated outputs for each uploaded image, allowing users to review detected defects clearly, even if processing takes a few seconds per image.

PIPELINE FLOWCHART



Thank You